

LLMOrchestrator

LLMOrchestrator

适用于所有无界面 CLI 编码代理的统一控制平面

概要

LLMOrchestrator 是一个独立且可复用的 Go 模块，用于生成、管理并与无界面 CLI 代理（OpenCode、Claude Code、Gemini、Junie、Qwen Code）进行通信。采用混合管道+文件协议，具备单代理熔断机制、可插拔的多供应商选择、解耦的国际化抽象层，并确保防欺诈测试保障。

简要说明

一个可复用的 Go 模块，通过混合管道+文件协议，为多个基于 LLM 的 CLI 代理提供统一接口，实现生成与驱动。支持线程安全的代理池，配备熔断机制及可选路由策略，设计上严格面向消费者无关性，并提供可插拔的国际化翻译器。

详细说明

LLMOrchestrator 是用于编排无界面 CLI 编码代理的共享基础设施——每个多代理系统背后都需要的基础架构，但通常重复构建且效果不佳。它无需每个项目为 OpenCode、Claude Code、Gemini CLI、Junie 或 Qwen Code 等工具重新实现进程生成、消息封装及结果解析，而是提供统一的 Agent 接口、线程安全的 AgentPool 以及 MultiProviderPool，后者将来自多个供应商的代理整合至单一门面。路由通过 AgentSelector 实现可插拔：可选择轮询模式（跳过不满足要求的供应商）或优先级顺序（带降级策略），工作分配方式由用户自定义，而非硬编码假设。每个具体代理均基于共享的 BaseAdapter 构建，后者全权管理进程生命周期：从管道建立、优雅的 SIGTERM-then-SIGKILL 停止、重启到存活检测——这些繁琐且易出错的部分，一次性解决。

通信采用混合模式，按需匹配传输方式。管道传输承载换行符分隔的 JSON 消息，每次请求设有读取超时及响应长度限制，适用于快速交互；文件传输则通过每会话的收件箱/发件箱/共享目录处理大型或持久化数据，避免管道承载过重。弹性设计并非事后补救，而是内置结构：单代理熔断器在连续三次

失败后开启，进入 60 秒冷却期，随后进行半开探测；后台健康监控主动 ping 代理，确保宕机代理无需等待流量发现即可恢复。代理池获取通过条件变量阻塞，避免 CPU 空转，响应解析器则无状态且支持并发调用。模块严格解耦——不允许任何消费者特定逻辑泄露其中——所有面向用户的字符串均通过可插拔的国际化 Translator 处理，默认提供 NoopTranslator，直接返回消息 ID，确保缺失翻译时能清晰暴露问题而非隐藏。

为何构建此模块

每个多代理系统都需要可靠地启动并与 CLI 代理通信。在每个项目中重复解决进程生成、消息封装、解析及故障处理既低效又易出错。

LLMOrchestrator 将这些功能集中至一个解耦、可复用的模块，其专注的职责确保了复用性——而一旦消费者特定逻辑侵入，复用性将不复存在。

内容

为何这是一场变革

它将“运行一支异构 CLI 代理军团”从每个项目定制化的工程噩梦，转变为一个简单的库导入——资源池管理、熔断机制、生命周期控制以及可插拔路由均已解决并经过实战验证。由于其反虚假测试（Anti-Bluff Tests）是对真实系统进行端到端验证，而非仅仅满足于“能编译通过”，因此你获得的是一个在并发与故障场景下真正可靠的抽象层，而非仅仅在图表中看起来完美的设计。

创新之处

- 混合管道+文件协议 —— 交互式速度（JSON 行通过 stdin/stdout 传输，带读取超时与响应限制）兼顾 持久化文件交换（收件箱/发件箱/共享区），确保大型数据传输时无需在低延迟与持久性之间做取舍。
- 多供应商资源池与可插拔选择器 —— 单一接口整合多种 CLI 供应商，路由策略可按轮询或优先级顺序动态配置，而非硬编码固定。
- 单代理熔断器 + 后台健康监控 —— 自动降级与恢复（3 次失败 → 60 秒断开 → 半开探测），确保不稳定代理被隔离后可自动悄然恢复，无需人工干预。
- 非忙等待资源池 —— Acquire 通过 sync.Cond 阻塞，直至匹配且健康的代理空闲或上下文取消，等待过程零 CPU 消耗。
- 严格解耦 + 反虚假国际化 —— NoopTranslator 原样返回消息 ID，确保缺失翻译绝无可能被忽略，而非悄然留白。
- 默认安全 —— 二进制路径白名单杜绝 shell 注入风险，并通过路径遍历防护、1 MiB 响应限制（防止输出失控）以及日志中 API 密钥屏蔽等措施，构建安全底线。
- 反虚假挑战测试框架 —— 跨五种语言（en/sr/ja/es/de）进行真实磁盘/JSON/解析器的完整往返测试，并配备变异检测门（LLMORCH_MUTATE_RUNNER=1 必须失败 → 包装器退出码 99），以验证功能确实可被测试捕捉到故障。

最大技术挑战及解决方案

- 可靠的代理进程 **I/O** 交互。与生成的 CLI 进程通信看似简单，实则充满陷阱。解决方案：采用混合管道+文件传输协议，通过明确的消息/解析器契约确保双方对通信格式达成一致，并由 BaseAdapter 集中管理进程全生命周期，包括优雅的 SIGTERM 超时机制，必要时升级为 SIGKILL 强制终止。
- 无忙等待的并发控制。解决方案：基于互斥锁 + 条件变量的 AgentPool，Acquire 休眠等待，直至匹配能力的代理真正空闲，配合无状态、无副作用的解析器，确保多 goroutine 并发调用的安全性。
- 供应商故障隔离。解决方案：单代理熔断器限制故障影响范围，后台健康监控 goroutine 驱动恢复，即使无请求触发也能自动修复。
- 验证正确性而非仅编译通过。解决方案：引入挑战测试框架（Challenge Runner），在 en/sr/ja/es/de 等语言环境下验证数十项不变性，并通过变异检测门（LLMORCH_MUTATE_RUNNER=1 必须失败 → 包装器退出码 99）故意破坏功能，以证明检测机制本身并非虚假。
- 本地化无静默失败。解决方案：通过 NoopTranslator 的消息 ID 原样返回机制，结合每个消费者可注入的翻译器，确保翻译缺失始终可见，而非被掩盖。

内容

技术栈

- **Go (1.25版)** ——选用此版本是因其具备一流的并发处理能力与简洁的进程控制，这正是协调实时代理进程所需的核心特性；该版本实现了模块本身、代理适配器、传输层及解析器。
- 仅 **Go** 标准库（外加 **testify**、**yaml.v3**）——有意为之，旨在最大限度缩小依赖范围，且不引入任何 LLM SDK，确保模块保持轻量化，可无缝嵌入任何使用方，无需附带供应商冗余。
- 管道传输（基于 **stdio** 的 **JSON-lines**）——选用此方式以实现快速交互式消息传递，并通过读取超时与响应长度限制强化安全性，确保僵死或失控的代理进程无法阻塞调用方。
- 文件传输（收件箱/发件箱/共享区）——针对每个会话中的持久化、大型工件交换场景设计，此处管道并非合适工具。
- **sync.Mutex/sync.Cond**——用于实现无忙等待的阻塞式、公平的代理池获取机制。
- 熔断器 + 健康监控器——两者结合，不仅提供单个代理的弹性恢复能力，还实现主动故障修复，而非仅停留于故障检测。
- **pkg/i18n** 翻译器——作为解耦的本地化接口，将特定于使用方的字符串与核心模块隔离。
- 测试框架（**challenges/runner**）+ **Makefile**（**test -race**、**fuzz**、**cover**）——采用此组合以确保验证过程基于实证而非假设，包含竞态检测与解析器模糊测试，在对抗性条件下验证代码正确性。

状态与诚信说明

- 状态：测试版。作为解耦可复用模块，已被多个 Helix/vasic 项目以子模块形式引用。许可证：**Apache-2.0**；GitHub 代码库已公开。
- 模型元数据通过 HelixQA 从 LLMsVerifier 桥接而来；本模块不直接引入 LLMsVerifier/VisionEngine/DocProcessor。父应用的 CLAUDE.md（如 Gin/PostgreSQL 等）中提及的技术栈描述的是 helix_code，而非本模块。

优先级：Helix 主要（LLM 基础设施集群——解耦可复用模块）。优先级低于 HelixTrack。